

Industry Daily Report — Methodology v2 & Source Solutions

Supersedes the 25 Aug methodology. Pipeline as deployed 26 Aug 2026, plus a solution evaluated for every link that has ever failed — with the option that wins, and why.

Part 1 — How the report is produced now

The clock

WHEN (AEST)	WHAT RUNS	WHAT IT DOES
Every hour	Source Ingest	Every active source, down a five-tier ladder; every tier attempt logged per run, so "tried and failed" is never confused with "never tried"
Every 30 min	Article Analysis	Scores new articles 0–100 (exits instantly when there is nothing new, so the cadence costs nothing)
06:00 · 12:00 · 23:00	News Quality	Opens undated articles and reads the publisher's own date; judges news from standing pages; writes the coverage line. The 23:00 pass completes the day before it closes
05:00	Daily Report	Assembles the finished prior day and sends via Microsoft Graph

The retrieval ladder — cheapest tier that works, per source, per run

TIER	ROUTE	WHAT IT DEFEATS	EVIDENCE
0	Email — newsletters into the intake mailbox	Account walls no fetcher can pass	79 newsletters received; see Part 2 for the one step still broken
1	Scrapling — Chrome TLS fingerprint	Fingerprint 403s	DCCEEW, S&P, EcoGeneration, NABERS: 403 → 200
1b	Sitemap — the publisher's XML, read as a feed	Client-rendered listings behind an edge block	S&P: 21 articles on day one, incl. every story the client flagged
2	Chromium render	Ordinary JavaScript pages	Standing fallback
3	Authenticated browser session	Member portals	Built for the CEC portal; awaiting the credential
4	Hermes (local 8B model) reads the page	Markup no parser recognises	Last resort, bounded

Dating — the fix for "missing" articles that were never missing

Seven sources in ten publish no machine-readable date, so articles were dated by when we happened to fetch them — and one in three landed in the wrong day, falling between two reports. Now: the date is read from the URL path at ingest, then from the article page itself (`article:published_time` , JSON-LD, `<time>`) by the quality passes, politely per host — because fetching too fast earns a 429 that is indistinguishable from "no date" (23 of 31 "dateless" articles were really rate-limited). Every page is visited once (`date_state`), a 429 is retried, a genuine no-date is not. An article whose resolved date proves it old is held automatically, so archive material can never masquerade as today's news again (484 were caught doing exactly that).

Judgement — three verdicts per article

- Relevance 0–100. Everything with any energy, building or climate bearing goes to the client (floor 1). A 0 means the wrong subject entirely — father's-day gadget guides — and stays in the database, dashboard and Teams list, never the email.
- News or reference. Hermes separates stories from standing pages ("Integrated System Plan", "Gas Retail Markets", a scholarship). Ten of ten correct on live titles; 33 pages held so far. Anything it will not commit on ships anyway — dropping a real story is the failure being fixed.
- Furniture. No date anywhere on the page *and* nothing relevant in it ("Legal notice and disclaimer") — never an article, excluded.

Assembly, sending, and the guarantees

The window runs on the real publication date, from two days back to now — wider than a day on purpose, because the ledger (`report_items` against sent reports) guarantees nothing repeats and nothing is lost to midnight. Verified: 0 duplicates. Send is Microsoft Graph from the tenant mailbox (Microsoft-signed; the old SMTP path failed DKIM and Gmail discarded it silently). Any non-202 throws, so a failed send can never be recorded as sent — this guard caught the one send failure since (a recipient-format bug, fixed same day). Every quality pass writes a coverage line: `fetches · dated · undated · wrong-day-corrected · sources returning zero`.

Part 2 — Every failing link, every option, and the winner

The one finding that changes the priority list: the newsletter route is 90% working and 0% delivering. 79 newsletters have arrived — Reuters (five briefings), The Australian, The Conversation — senders map correctly and links are extracted (107 in one Conversation issue, 102 in one Australian issue). But articles kept: zero, on every message. Cause found in the keep step: newsletter links are tracking-wrapped (they point at the mail domain, e.g. `e.theaustralian.com.au`, not the publisher), and the judge filters to same-host links before unwrapping — so every real story is discarded as "not the publisher's". The fix is to unwrap the tracking redirect *before* judging. One code change; it unlocks Reuters and The Australian at a stroke.

Solved — no further spend or effort needed

LINK / SOURCE	FAULT	OPTIONS WEIGHED	WINNER
DCCEEW · dcceew.gov.au	403 to every ordinary fetch	Scrapling ✓ · paid proxy (unneeded) · Oxylabs (unneeded)	SOLVED Scrapling TLS impersonation — 28 articles/wk
EcoGeneration	Same 403 class	Scrapling ✓ + native RSS ✓	SOLVED RSS primary — 11/wk
NABERS	403 + program pages captured as news	Scrapling ✓ + Hermes judge ✓	SOLVED both — 34/wk, furniture held
S&P latest-news & crude-oil	Client-rendered listing + edge 403 to Chromium — <i>no fetcher could ever work</i>	Scrapling ✗ (empty shell) · Chromium ✗ (403) · Oxylabs render (would work, paid) · sitemap ✓ (free)	SOLVED XML sitemaps — 21 articles day one, incl. the flagged stories
Renewables Now	429 rate-limit under parallel fetch; no dates in markup	Per-host politeness ✓ · proxy rotation (unneeded, impolite)	SOLVED politeness + article-page dates — 167/wk, top source
AEMO / GEMS / AER pages	Navigation and explainers captured as news	Regex rules ✗ (defeated) · Hermes judge ✓	SOLVED the model — 10/10 on live titles

One fix away — the highest-value actions, all free

LINK / SOURCE	FAULT	OPTIONS WEIGHED	WINNER & ACTION
Reuters · reuters.com	Login (401). No fetcher, proxy or rendering service passes a genuine login — Oxylabs and Firecrawl included	Scraping ✗ · Oxylabs ✗ (login) · newsletter ✓ arriving (5 briefings, 149 links found)	FIX KEEP STEP unwrap tracking links before the judge
The Australian	Subscription site	Scraping ✗ · newsletter ✓ arriving (102 links/issue)	FIX KEEP STEP same one change
IEA · iea.org	Login (403). Retired at 527 straight failures — visible, not hidden	Scraping ✗ · Oxylabs ✗ · newsletter — not yet arriving (absent from senders)	SUBSCRIBE confirm the IEA newsletter into the intake mailbox
AFR article bodies	Listing scrapes fine (38/wk); bodies paywalled	Oxylabs ✗ (paywall) · newsletter (partial) · tier-3 session ✓ — the operator holds a login	TIER-3 CREDENTIAL use the existing session-fetch route
The Conversation foreign editions	FR/ES/US/global newsletters arriving as noise beside a working AU scrape	Keep ✗ (noise) · filter to AU ✓	UNSUBSCRIBE / FILTER non-AU editions
CEC member portal	Member-only content (Monday Megawatt)	Tier-3 login ✓ built · nothing else reaches it	CREDENTIAL PENDING + check licensing before client use

Genuinely open — and what each would cost to close

LINK / SOURCE	FAULT	OPTIONS, IN COST ORDER	RECOMMENDATION
vic-premier · premier.vic.gov.au	Articles load by in-page XHR; HTML carries no links. 428+ visible failures	1. Capture the JSON endpoint in browser dev-tools — free, ~30–60 min, then it is just another feed 2. Firecrawl self-host pilot — free software, one more container 3. Oxylabs render — paid	OPEN option 1 first; it is the entire fix if the endpoint is stable
aer-registers · aer.gov.au	Same class: client-rendered register, no feed	Same three, same order	OPEN same session as vic-premier
S&P natural-gas	Sitemap fetches (1.1 MB) but yields nothing recent	1. Verify it is the right sitemap (minutes) 2. Register electric-power / LNG sections the proven way	VERIFY quiet vs wrong file — not yet proven either way, and said so

Part 3 — The platforms, measured against what is actually left

Oxylabs (dashboard.oxylabs.io) — does it solve these?

Mostly no, because most of it is already solved — and the rest it cannot solve. Oxylabs sells proxies, a scraper API and headless rendering. Against this pipeline's history: TLS-fingerprint 403s — solved free by Scrapling. Rate limits — solved free by politeness (and rotating IPs to evade a publisher's rate limit is the impolite version of the same thing). Client-rendered pages — its rendering *would* work, but that class is down to two endpoints, each likely a one-hour manual capture. Logins and paywalls (Reuters, IEA, AFR, The Australian) — no scraping service passes a genuine login; the newsletter and session routes are the lawful answer and are nearly done. Dating, judgement, send, monitoring — pipeline problems no scraper vendor touches.

Verdict: not now. A subscription would buy convenience for a two-endpoint problem. Revisit only if the source list grows to many JS-heavy sites at once — and pilot self-hosted Firecrawl (free) before paying for the same capability.

Where each tool is the right tool

TOOL	BEST AT (PROVEN HERE)	WRONG FOR	COST
Scrapling (running)	TLS-fingerprint blocks; fast polite page fetches — the workhorse of every tier-1 fetch and every date read	JavaScript rendering; logins	Free, self-hosted, already deployed
Sitemap reading (running)	Client-rendered sites whose XML is served openly — better dates than the listing ever had	Sites without sitemaps	Free
Hermes agent (running)	Editorial judgement: news vs reference (10/10), article-link extraction from unrecognised markup, the newsletter keep decision	Bulk work — 7.5 tokens/sec means per-article use risks the 05:00 deadline; batches above 5 drift out of format	Free, local
Newsletters (tier 0)	The only lawful route past a login — Reuters, IEA, The Australian	Completeness (a daily issue carries a fraction of the day)	Free; one keep-step fix outstanding
Firecrawl (pilot candidate)	JS rendering + crawl for the XHR class, self-hosted	Replacing the working stack; logins	Free software; ~1–2 GB RAM on a VPS already running an 8B model — check headroom
Oxylabs (deferred)	Scale: many JS-heavy or geo-blocked sources at once	Logins/paywalls; problems already solved free	Paid subscription

The decision rule the pipeline follows

For any link: the cheapest tier that demonstrably works, verified by evidence, visible when it fails. Email beats scraping where a login stands. A sitemap beats a browser where one is served. A parser beats a model where markup is regular; the model earns its place only where the call is editorial. A paid service earns its place only when the free options are exhausted — and after a week of failures, none of the free options is exhausted anywhere except two Victorian XHR endpoints.

Priority order, by articles gained per hour of work

- 1. Fix the newsletter keep step (unwrap tracking links before judging) — unlocks Reuters and The Australian in one change; the mail is already arriving.
- 2. Subscribe the IEA newsletter — the one walled source with no mail arriving at all.
- 3. Capture the two XHR endpoints (vic-premier, aer-registers) — closes the only class no current tier reaches.
- 4. Verify S&P natural-gas and register electric-power / LNG — extends a proven route.
- 5. AFR tier-3 session + CEC portal credential — full bodies and member content.
- 6. Wire the quiet-source count to Teams — the next silent failure pages us, not the client.

Part 4 — The playbook: one source showing all four faults

The four faults of the 25 Aug diagnosis are no longer things a person fixes per source — three of the four are now structural: the machinery fires for every source automatically, old and new alike. What follows is what fires on its own, what a person can force when they will not wait for the schedule, and the escalation when the automatic answer is not enough.

Fault 1 — articles undated, or filed under the wrong day

AUTOMATIC (ALREADY FIRING)	FORCE IT NOW, BY HAND	ESCALATE WHEN
At ingest, the date is read from the URL path. What is still undated is opened by the quality passes (06:00/12:00/23:00) — the article page is fetched politely via Scrapling and <code>article:published_time</code> / JSON-LD read. A 429 is marked <code>blocked</code> and retried; each page is visited once (<code>date_state</code>).	<code>node sources/backfill-dates.js</code> — the same logic, immediately, over the backlog. Or execute BEXT — News Quality in n8n.	The page truly carries no date (CEC, NABERS style). Then the judge decides what it is; a real story stays, dated by first-seen, and the coverage line counts it honestly.

Fault 2 — the midnight drift

AUTOMATIC	FORCE	ESCALATE
Nothing to do per source — this fix is entirely structural. The window runs on the real publication date, two days back to now; the ledger guarantees an article appears exactly once and can never fall between two reports; the 23:00 pass closes the day before the report reads it.	Nothing to force. (Verified: 0 duplicates, 0 losses since deployment.)	Never — there is no per-source version of this fault any more.

Fault 3 — captures that are not articles

AUTOMATIC	FORCE	ESCALATE
Three gates before the sheet: the furniture rule (no date anywhere <i>and</i> relevance 0); the Hermes judge (news vs standing page — 10/10 on live titles, undecided ships); the stale-hold (a resolved date older than 14 days holds the article the moment it is known).	<code>node sources/classify-kind.js -days 5</code> — judge the current window immediately.	The judge keeps abstaining on a source's pages → tighten that source's link scoring in the parser (the permanent fix), using the held/abstained lists as the worked examples.

Fault 4 — the source goes quiet and nobody is told

AUTOMATIC TODAY	FORCE	ESCALATE — THE DIAGNOSIS LADDER
Every tier of every fetch is logged per run; the coverage line counts sources that ran clean and returned zero; the dashboard shows the tier strip and "exhausted every route". The remaining gap: no Teams page yet — priority 6 in Part 3.	Execute BEXT — Source Ingest in n8n (the force-fetch — all tiers, logged), then read <code>fetch_attempts</code> for that source.	What the attempt log says decides the tool — this is the S&P procedure, generalised: · 403 / TLS refusal → Scrapling tier 1 (automatic) · 200 but zero links → client-rendered: check <code>robots.txt</code> for a sitemap → register method: <code>sitemap</code> (proven on S&P) · Login wall → newsletter (tier 0) or session login (tier 3) — no scraper or proxy service passes a genuine login · Markup no parser reads → Hermes tier 4 extracts the links (its judgement job, not bulk) · Still nothing → capture the XHR endpoint by hand, or the Firecrawl pilot — the last free rungs before any paid service

Which model does what — the division of labour

Gemini (cloud, quota-bound) does the one big-volume judgement: relevance scoring, in batches, every 30 minutes. Hermes (local, free, slow) does the small editorial judgements where rules fail: is this link an article, is this page news or reference, is this newsletter item worth keeping. Neither model fetches anything — retrieval is deterministic ladder code, because the failure being engineered out is *invisible* failure, and a model deciding what to fetch is unauditable. The agent judges; the ladder fetches; the log remembers; the coverage line tells on both.